Arquitetura do ArangoDB e AQL

Nov 2, 2026 · @Arthur

Material da Parte 3 do Seminário NoSQL do Grupo 5: cerca de 5 minutos, divididos em 5 slides de aproximadamente 1 minuto cada. Cada seção abaixo traz o que vai no slide e o que falar. Os exemplos usam as coleções da demo: `usuarios`, `amizades` e `sessoes`.

Slide 1 — Modelo de dados: coleções e documentos

No ArangoDB, tudo é documento JSON guardado em coleções, e o grafo nasce de um tipo especial de coleção: a de arestas.

No slide:

- Coleção de documentos: guarda os vértices (`usuarios`, `sessoes`)
- Coleção de arestas (edges): guarda as ligações (`amizades`)
- Campos automáticos: `_key`, `_id`, `_rev`, mais `_from` e `_to` nas arestas

```
// documento em usuarios
{ "_key": "u01", "_id": "usuarios/u01", "_rev": "_hX3k...", "nome": "Ana" }

// aresta em amizades
{ "_from": "usuarios/u01", "_to": "usuarios/u02", "desde": 2024 }
```

Campo	O que guarda	Quem define
`_key`	Chave única dentro da coleção	Você ou o banco; imutável
`_id`	`coleção/chave`, único no banco inteiro	Banco, a partir de `_key`; imutável
`_rev`	Revisão (versão) do documento	Banco, a cada alteração
`_from` / `_to`	`_id` dos vértices de origem e destino	Você, só em arestas

O que falar: todo documento recebe atributos de sistema que começam com underscore. O `_key` pode ser escolhido na criação; o `_id` é montado a partir da coleção e da chave, e nenhum dos dois muda depois. O `_rev` é mantido pelo próprio banco (fonte).

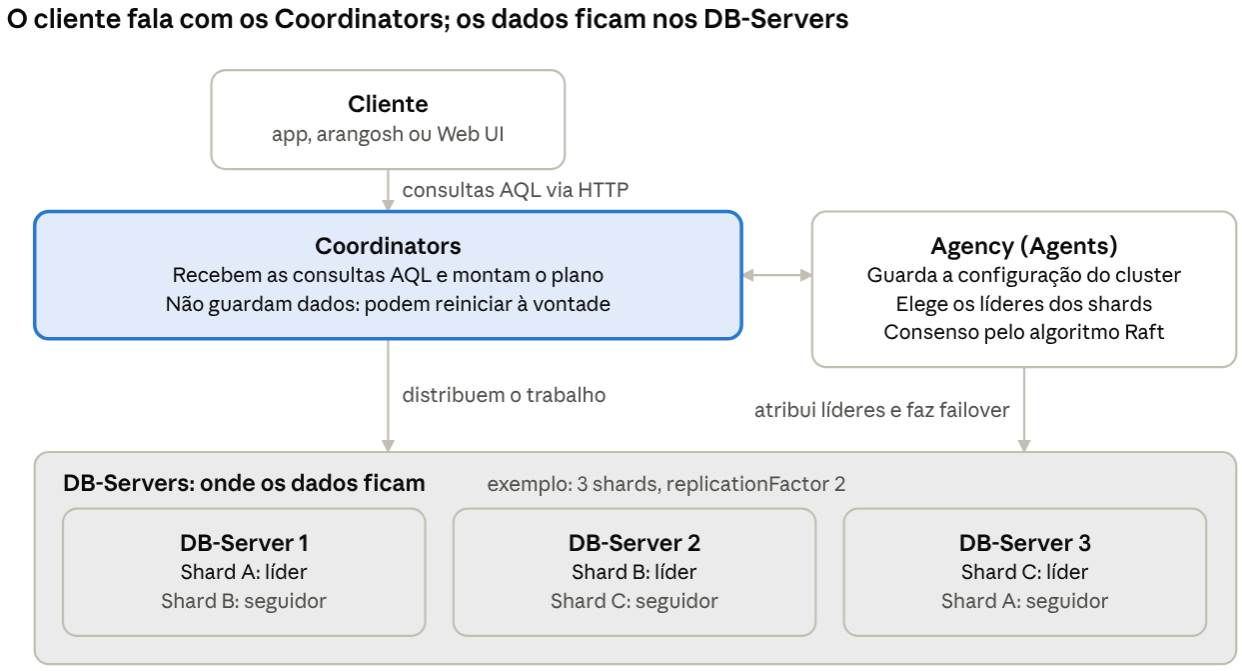
Gancho de desempenho: `_id`, `_key`, `_from` e `_to` já vêm indexados. Toda coleção de arestas ganha um edge index automático sobre `_from` e `_to`, e é isso que torna a travessia de grafo rápida (fonte).

Slide 2 — Arquitetura: servidor único x cluster

O ArangoDB roda como servidor único (o modo da nossa demo) ou como cluster, em que cada processo assume um de três papéis.

No slide:

- Servidor único: um processo só, simples de subir
- Cluster: Agents, Coordinators e DB-Servers
- Modelo CP (consistência + tolerância a partição), sem ponto único de falha



arquitetura do cluster · 3 papéis

O cliente nunca fala direto com os DB-Servers: tudo passa pelos Coordinators, e a Agency mantém o mapa de quem é líder de cada shard.

O que falar: a arquitetura de cluster segue um modelo CP master/master sem ponto único de falha (fonte).

- **Agents:** formam a Agency, o lugar central da configuração do cluster. Ela faz eleição de líderes e, sem ela, nenhum outro componente funciona. Para tolerar falhas, os Agents usam o algoritmo de consenso Raft (fonte).
- **Coordinators:** são os únicos acessíveis de fora. Recebem as consultas, coordenam a execução e devolvem o resultado. Não guardam estado, então podem ser reiniciados livremente (fonte).
- **DB-Servers:** hospedam os dados em shards; para cada shard, um DB-Server é líder ou seguidor (fonte).

Slide 3 — Shards e replication factor

Shards dão escalabilidade horizontal; o replication factor dá alta disponibilidade.

No slide:

- Shard: pedaço de uma coleção espalhado entre DB-Servers
- Replication factor: quantas cópias de cada shard existem (líder + seguidores)
- Se o líder cai, um seguidor assume (failover automático)

```
db._create("usuarios", { numberOfShards: 4, replicationFactor: 3 })
```

Conceito	Para que serve	Padrão
`numberOfShards`	Dividir a coleção entre máquinas	1 shard
`shardKeys`	Atributo usado no hash que escolhe o shard	`_key`
`replicationFactor`	Número total de cópias de cada shard	1 (sem replicação)

O que falar: com sharding, várias máquinas formam um único banco, e o ArangoDB distribui os dados sozinho entre elas (fonte). Para escolher o shard de um documento, ele calcula um hash, por padrão sobre o `_key` (fonte).

O replication factor conta o líder junto com os seguidores: valor 3 significa um líder e dois seguidores, ou seja, o dado existe três vezes. As escritas vão primeiro ao líder e são replicadas de forma síncrona. O padrão é 1, que na prática desliga a replicação (fonte); se a única máquina com o dado falhar de vez, ele se perde (fonte).

Gancho com grafos: a documentação recomenda o modo OneShard para a maioria dos casos de grafo. Nele, cada coleção fica com um único shard e todos os líderes ficam no mesmo DB-Server, evitando idas e vindas pela rede durante a travessia (fonte).

Slide 4 — AQL: estrutura básica e comparação com SQL

AQL é declarativa como SQL, mas escrita como um laço: o `FOR` percorre os documentos e cada linha seguinte filtra, ordena ou transforma.

No slide:

```
FOR u IN usuarios
  FILTER u.cidade == "Recife"
  SORT u.nome ASC
  LIMIT 5
  RETURN { nome: u.nome, idade: u.idade }
```

SQL	AQL
`SELECT ... FROM usuarios`	`FOR u IN usuarios ... RETURN u`
`WHERE`	`FILTER`
`ORDER BY`	`SORT`
`LIMIT`	`LIMIT`
`JOIN`	`FOR` dentro de `FOR`
`GROUP BY`	`COLLECT`
`INSERT INTO`	`INSERT ... INTO`
`UPDATE ... SET`	`UPDATE ... WITH ... IN`
`DELETE FROM`	`REMOVE ... IN`

O que falar: `FOR ... IN` faz o papel do `FROM`, `FILTER` do `WHERE` e `RETURN` do `SELECT`. A diferença principal é que o `RETURN` monta qualquer JSON de saída, não só colunas. E a mesma linguagem serve para documento, chave-valor e grafo, que é o argumento central do multimodelo.

Exemplo de JOIN, para mostrar o laço aninhado:

```
FOR s IN sessoes
  FOR u IN usuarios
    FILTER u._key == s.usuario
    RETURN { usuario: u.nome, token: s._key }
```

Slide 5 — Travessia de grafo em AQL

"Amigos de amigos" vira uma linha: no relacional seriam dois JOINs na tabela N:N; aqui basta mudar a profundidade.

No slide:

```
FOR v, e, p IN 1..2 ANY "usuarios/u01" amizades
  RETURN DISTINCT v.nome
```

Parte	Significado
`v`, `e`, `p`	Vértice atual, aresta atual e caminho percorrido

Parte	Significado
`1..2`	Profundidade mínima e máxima (amigos e amigos de amigos)
`OUTBOUND` / `INBOUND` / `ANY`	Segue arestas de saída, de entrada ou as duas
`"usuarios/u01"`	Vértice de partida, pelo `_id`
`amizades`	Coleção de arestas percorrida

O que falar: o `FOR` da travessia pode devolver até três variáveis: vértice, aresta e caminho (fonte). No `min..max`, o mínimo padrão é 1, e se o máximo for omitido ele vale o mesmo que o mínimo (fonte). Como amizade é simétrica, usamos `ANY`. Para ir mais fundo, é só trocar `1..2` por `1..3`; no SQL isso exigiria mais um JOIN ou uma CTE recursiva.

Rodapé do slide: o painel mostra "Enterprise Edition" porque, a partir da versão 3.12.5, a Community Edition inclui todos os recursos da Enterprise sem limite de tempo (fonte). Continua sendo necessária licença para uso comercial ou para mais de 100 GiB de dados (fonte).

Dicas e possíveis perguntas

- **Query do slide 5 ao vivo:** inclua `FILTER v._key != "u01"` antes do `RETURN`, porque o próprio usuário inicial pode aparecer no resultado. O `DISTINCT` evita nomes repetidos quando um amigo direto também é amigo de amigo.
- **"Onde os dados ficam gravados?"** O motor de armazenamento é o RocksDB, baseado em LSM-tree e otimizado para escrita.
- **"A demo roda em cluster?"** Não: a demo usa servidor único. O cluster é o caminho para escalar quando o volume ou a disponibilidade exigem.
- **"O que significa CP?"** No teorema CAP, o cluster prioriza consistência e tolerância a partição.
- **Curiosidade:** a documentação da versão 4.0, ainda em desenvolvimento, passou a chamar vértice de "node" na sintaxe de travessia (fonte).

Referências

- Cluster: arquitetura e papéis
- Administração do cluster: sharding e replicação
- OneShard
- Índices: primary index e edge index

- Atributos de sistema dos documentos
- Travessias de grafo em AQL
- Edições do ArangoDB (Community x Enterprise)
- Novidades da versão 3.12